ROHITH CHADUVALA

Full-Stack AI Engineer

+91 8885405754 | chaduvalarohith@gmail.com

GitHub • LinkedIn • LeetCode

EDUCATION

Anurag University, Hyderabad (2022 – 2026), B.Tech CSE, CGPA: 8.35 (7th Sem)
Sri Kakatiya jr. college, Godavarikhani (2020 – 2022), 11th–12th (PCM), 96.2%
Krishnaveni talent school, Godavarikhani (2010 – 2020), till 10th, 10/10 CGPA

EXPERIENCE

Programmer Analyst Trainee — Cognizant

Feb 2026 – Present

GenC — QE(QEA) | Chennai (Siruseri), India

- **Foundation & Automation:** Completed Core Java, SQL, and manual testing fundamentals (SDLC, STLC, Agile/JIRA). Progressed into Selenium WebDriver with dynamic XPath, POM framework, Apache POI for data-driven testing, and TestNG for structured test execution and reporting.
- **Projects & Current Work:** Built an individual mini-project automating *Caribbean Cruisers* end-to-end business flows using Selenium + TestNG. Working in a team on a Hackathon project automating *MakeMyTrip* with POM and Extent Reports. Currently progressing through REST API testing using Postman and RestAssured; additionally covered Git, Maven, Jenkins, Docker, and BDD Cucumber as part of the program.

TECHNICAL SKILLS

Languages:	Python, Java, JavaScript, SQL, HTML / CSS
Frontend & Frameworks:	React, FastAPI, Node.js, Express, REST APIs
AI & Generative AI:	RAG Pipelines, LangGraph (Agentic Workflows), AutoGen, Prompt Engineering, OpenAI API, LLMs & Embeddings, Multi-Agent Systems
AI Coding & Evaluation:	GitHub Copilot, Claude, RAGAS, ChromaDB, Sentence-Transformers, Structured JSON Logging, AI Guardrails, Fallback & Degradation Strategies
Cloud & DevOps:	Microsoft Azure (Functions, Event Hubs, Fabric, Databricks, ADF, Synapse), PySpark, Snowflake, Power BI, ETL Pipelines, Bronze-Silver-Gold Architecture
Databases:	MySQL, PostgreSQL, MongoDB, SQLite
Testing & QE:	Selenium WebDriver, TestNG, Page Object Model, Apache POI, Maven, Extent Reports

PROJECTS

PrescribeCheck — RAG-based Drug Interaction & Side-Effect Checker

GitHub

React • JavaScript • FastAPI • Python • ChromaDB • Sentence-Transformers • OpenAI • Pydantic • Docker • GitHub Actions

- **RAG Pipeline over FDA Drug Labels:** Built a full Retrieval-Augmented Generation pipeline over a curated corpus from the openFDA API (drug interactions, warnings, and adverse reactions across ~480 essential medicines) stored in ChromaDB with sentence-transformer embeddings. Retrieval combines drug-pair metadata filtering with semantic similarity search before prompting OpenAI, returning grounded answers with source-section citations visible in the UI
- **Structured Outputs & Section-Aware Chunking:** Designed Pydantic schemas (InteractionVerdict with severity, effect, evidence chunks, and confidence) ensuring the LLM returns deterministic JSON instead of free text. Implemented section-aware chunking that splits FDA labels by their numbered subsections (7.1, 7.2, ...) preserving table structure and clinical context across drug-pair retrievals.

- **AI Guardrails & Fallback Strategy:** Implemented a citation-ID verification layer that rejects LLM responses citing chunk IDs outside the retrieval set, an out-of-scope handler returning graceful "no data" responses for unknown drugs instead of hallucinated answers, and mandatory medical disclaimers on every output following responsible AI practices.
- **FastAPI Backend & React Frontend:** Built a FastAPI backend exposing /api/check with Pydantic request validation, async LLM calls, and structured JSON logging (request_id, drugs, latency_ms, retrieval_hit_count) for observability. The React + JavaScript frontend renders severity-coded results, an expandable retrieved-chunks panel for source transparency, and clean loading and error states.
- **Dockerized Microservices:** Containerized the backend with Docker and a persistent ChromaDB volume; configured a GitHub Actions workflow running pytest and ruff on every pull request. Backend deployed on Render with auto-deploy on main branch; React frontend deployed on Vercel with environment-based API routing.

Agentic DSA Solver — Multi-Agent Code Generation & Validation GitHub

Python • LangGraph • AutoGen • Pydantic • Docker • OpenAI API • Streamlit

- **Multi-Agent Graph with LangGraph:** Built a stateful LangGraph agent graph with Planner, Coder, and Executor roles collaborating to solve DSA problems; conditional edges route between agents based on execution outcomes, with retry logic and a recursion guard preventing infinite loops on unsolvable cases.
- **Sandboxed Execution & Structured Outputs:** Designed a Docker-based execution sandbox that runs LLM-generated solutions in isolation and feeds error traces back to the Coder agent for self-correction; agents communicate through validated Pydantic schemas (ProblemSpec, SolutionDraft, ExecutionResult) ensuring deterministic state transitions.
- **Prompt Engineering & AutoGen Benchmark:** Crafted role-specific prompts with chain-of-thought reasoning, code-generation guidelines, and tool-calling patterns for the sandbox; built a parallel AutoGen Round-Robin implementation alongside the LangGraph version to benchmark orchestration overhead and message verbosity.

WeatherPulse — AI-Augmented Real-Time Weather Intelligence Pipeline GitHub

Azure Functions • Databricks • PySpark • Azure Event Hubs • Microsoft Fabric (KQL DB) • Power BI • OpenAI API

- **Real-Time Event-Driven Pipeline:** Built an Azure Functions ingestion service polling WeatherAPI and publishing readings to Azure Event Hubs at one-minute cadence; PySpark transformations in Databricks apply schema validation, deduplication, and Bronze–Silver–Gold layering before feeding Microsoft Fabric KQL DB with under-5-second end-to-end latency.
- **LLM Anomaly Explainer:** Added an LLM-based anomaly summarizer triggered by Fabric Alerts on threshold breaches (high AQI or temperature spikes); recent readings are sent to OpenAI as context, returning plain-English root-cause hypotheses routed to ops stakeholders via email.
- **Power BI Dashboard & Cloud-Native Architecture:** Built a live Power BI dashboard for temperature, AQI, alerts, and short-term forecasts; designed the pipeline using Azure-native services with structured logging, retry policies, and Key Vault-based credential management.

CERTIFICATIONS

- Microsoft Certified: Azure Fundamentals (AZ-900) — Microsoft
- Lakehouse Fundamentals — Databricks
- Data Engineering on Microsoft Azure (DP-203) — Udemy
- Data Analytics & Visualization Job Simulation — Accenture
- Infosys Springboard — Machine Learning, Cloud Computing, Java, SQL, Big Data, Design Thinking